

V1 Supabase 준비물 획득 안내서

이 문서는 **phoenix pages**를 V1으로 올리기 전에 마스터가 직접 준비해야 할 Supabase, Google 로그인, Storage, 관리자 정보를 얻는 방법을 정리한 안내서이다.

V1의 목표는 다음과 같다.

1. Google 로그인 적용
2. Supabase Auth 사용
3. Supabase Storage에 생성 이미지 저장
4. Supabase Database에 프로젝트 정보 저장
5. 슈퍼 관리자와 일반 유저 구분
6. 브라우저 저장 중심 V0 구조를 클라우드 저장 중심 구조로 개선

0. 먼저 알아야 할 것

V0에서는 이미지와 작업 기록이 주로 브라우저에 남았다.

V1에서는 구조가 이렇게 바뀐다.

유저 로그인

- 이미지 생성
- Supabase Storage에 이미지 파일 저장
- Supabase Database에 프로젝트명, 유저, 이미지 URL 저장
- 유저는 자기 작업만 다시 열기
- 슈퍼 관리자는 전체 유저 작업 확인

따라서 V1 준비물은 단순히 API 키 하나가 아니라, 아래 묶음이다.

1. Supabase 프로젝트 정보
2. Supabase 공개용 키
3. Supabase 서버용 비밀 키
4. Supabase Storage bucket 이름
5. Google OAuth Client ID
6. Google OAuth Client Secret
7. Supabase Google Auth 설정
8. 슈퍼 관리자 이메일
9. 배포 환경변수 입력 위치

1. 최종으로 모아야 할 값

아래 표를 먼저 보고, 하나씩 채우면 된다.

구분	값 이름	어디에 쓰나	공개 가능 여부
Supabase	NEXT_PUBLIC_SUPABASE_URL	브라우저와 서버가 Supabase 프로젝트에 접속	공개 가능
Supabase	NEXT_PUBLIC_SUPABASE_ANON_KEY 또는 NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	브라우저에서 로그인, 읽기, 유저 권한 요청	공개 가능

구분	값 이름	어디에 쓰나	공개 가능 여부
Supabase	SUPABASE_SERVICE_ROLE_KEY 또는 SUPABASE_SECRET_KEY	서버에서 Storage 저장, 관리자 작업	절대 공개 금지
Supabase Storage	SUPABASE_STORAGE_BUCKET	이미지 저장 bucket 이름	공개 가능
Google OAuth	GOOGLE_CLIENT_ID	Google 로그인 연결	공개되어도 큰 문제는 낮음
Google OAuth	GOOGLE_CLIENT_SECRET	Google 로그인 서버 설정	절대 공개 금지
관리자	SUPER_ADMIN_EMAILS	슈퍼 관리자 이메일 목록	공개 권장 안 함
선택	DATABASE_URL	직접 Postgres 연결이 필요할 때	절대 공개 금지

권장 bucket 이름:

`SUPABASE_STORAGE_BUCKET=phoenix-pages-images`

권장 슈퍼 관리자 이메일 형식:

`SUPER_ADMIN_EMAILS=master@example.com`

여러 명이면 쉼표로 구분한다.

`SUPER_ADMIN_EMAILS=master@example.com,admin@example.com`

2. Supabase 프로젝트 만들기

이미 Supabase 프로젝트가 있다면 이 단계는 건너뛰어도 된다.

2.1 Supabase 접속

1. 브라우저에서 Supabase에 접속한다.
2. 로그인한다.
3. Dashboard로 이동한다.

공식 주소:

<https://supabase.com/dashboard>

2.2 새 프로젝트 생성

1. Dashboard 왼쪽 또는 상단에서 **New project**를 누른다.
2. Organization을 선택한다.
3. Project name을 입력한다.

추천 이름:

`phoenix-pages`

4. Database Password를 만든다.
5. Region을 선택한다.

한국 사용자가 많다면 가까운 아시아 리전을 우선 고려한다.

6. **Create new project**를 누른다.
7. 프로젝트 생성이 끝날 때까지 기다린다.

완료 기준:

Supabase 프로젝트 Dashboard에 접속 가능하다.
왼쪽 메뉴에 Table Editor, SQL Editor, Authentication, Storage, Project Settings가 보인다.

3. Supabase Project URL 얻기

이 값은 V1 코드에서 Supabase 프로젝트 주소로 사용한다.

3.1 메뉴 이동

1. Supabase Dashboard에서 V1에 사용할 프로젝트를 연다.
2. 왼쪽 아래 또는 왼쪽 메뉴에서 **Project Settings**를 누른다.
3. **API Keys** 또는 **API** 메뉴를 연다.
4. **Project URL** 또는 **URL** 항목을 찾는다.

값 모양:

`https://xxxxxxxxxxxxxxxxxxxxx.supabase.co`

3.2 기록할 이름

이 값을 아래 이름으로 기록한다.

`NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxxxxxxxx.supabase.co`

주의:

이 URL은 공개되어도 괜찮다.
하지만 URL만 있다고 보안이 완성되는 것은 아니다.
테이블 Row Level Security 정책이 반드시 필요하다.

4. Supabase 공개용 키 얻기

브라우저에서 Supabase Auth와 유저 권한 요청에 사용할 키다.

Supabase는 새 키 체계와 레거시 키 체계를 함께 제공한다.

새 프로젝트라면 가능하면 **Publishable key**를 우선 사용한다.

기존 코드나 라이브러리 호환 때문에 필요하면 **anon key**를 사용할 수 있다.

4.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. **Project Settings**로 이동한다.
3. **API Keys** 메뉴를 연다.
4. **Publishable key** 또는 **anon public key**를 찾는다.

새 키 모양:

sb_publishable_...

레거시 anon key 모양:

eyJhbGciOi...

4.2 기록할 이름

새 키를 쓰는 경우:

NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...

레거시 anon key를 쓰는 경우:

NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...

V1 구현 시 둘 중 하나를 기준으로 코드에 맞춘다.

주의:

공개용 키는 브라우저에 들어가도 되는 키다.
하지만 이 키만으로 아무 데이터나 열리면 안 된다.
반드시 Supabase RLS 정책으로 유저별 접근을 제한해야 한다.

5. Supabase 서버용 비밀 키 얻기

이 키는 서버에서만 사용한다.

용도:

1. Storage에 이미지 저장
2. 관리자 작업
3. 서버 API에서 권한 확인 후 DB 작업

절대 브라우저 코드에 넣으면 안 된다.

GitHub에 올리면 안 된다.

문서나 스크린샷에 노출하면 안 된다.

5.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. **Project Settings**로 이동한다.
3. **API Keys** 메뉴를 연다.
4. **Secret keys** 또는 레거시 **service_role** key를 찾는다.

새 Secret key 모양:

sb_secret_...

레거시 service_role key 모양:

eyJhbGciOi...

5.2 기록할 이름

새 Secret key를 쓰는 경우:

SUPABASE_SECRET_KEY=sb_secret_...

레거시 service_role key를 쓰는 경우:

SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...

V1에서는 서버 코드에서만 이 값을 사용한다.

주의:

service_role 또는 secret key는 RLS를 우회할 수 있는 강한 권한이다.
브라우저에서 사용하면 안 된다.
NEXT_PUBLIC_ 접두사를 붙이면 안 된다.
Vercel/Netlify 환경변수에는 서버 전용으로만 넣는다.

6. Supabase Storage bucket 만들기

생성된 상세페이지 이미지를 저장할 공간이다.

6.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. 왼쪽 메뉴에서 **Storage**를 누른다.
3. **Buckets** 화면으로 이동한다.
4. **New bucket** 또는 **Create bucket**을 누른다.

6.2 bucket 이름 입력

추천 이름:

phoenix-pages-images

6.3 Public / Private 선택

권장:

Private bucket

이유:

유저별 생성 이미지가 들어가기 때문에 기본은 비공개가 안전하다.
필요할 때 서버에서 signed URL을 만들어 보여주는 방식이 더 안전하다.

단, 빠른 테스트만 할 때는 Public bucket도 가능하다.

운영형 V1에서는 Private을 우선 권장한다.

6.4 파일 제한 설정

가능하면 아래처럼 제한한다.

Allowed MIME types: image/*
File size limit: 10MB 또는 운영 정책에 맞는 값

6.5 기록할 이름

SUPABASE_STORAGE_BUCKET=phoenix-pages-images

7. Supabase Database 준비

이 단계에서는 아직 테이블을 만들지 않아도 된다.

하지만 V1 코딩 작업을 시작하려면 어떤 테이블을 만들지 알고 있어야 한다.

V1에서 필요한 대표 테이블:

profiles
projects
project_sections
admin_users

역할:

테이블	역할
profiles	로그인 유저 기본 정보
projects	프로젝트 제목, 생성자, 채널, 상태
project_sections	각 상세페이지 이미지 URL, 섹션명, 프롬프트
admin_users	슈퍼 관리자 이메일 또는 user id

테이블 생성은 코덱스가 V1 작업 때 SQL로 진행하면 된다.

마스터가 지금 직접 해야 할 것은 Supabase 프로젝트와 권한 정보 준비다.

8. 선택: DATABASE_URL 얻기

V1을 Supabase JS SDK 중심으로 만들면 **DATABASE_URL**은 필수가 아닐 수 있다.

하지만 아래 경우에는 필요할 수 있다.

1. 서버에서 Postgres에 직접 연결할 때
2. 마이그레이션 도구를 쓸 때
3. 백업/관리 스크립트를 만들 때
4. Prisma, Drizzle 같은 ORM을 쓸 때

8.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. **Project Settings**로 이동한다.
3. **Database** 메뉴를 연다.
4. **Connection string** 또는 **Connection pooling** 항목을 찾는다.
5. URI 형식의 연결 문자열을 복사한다.

값 모양:

```
postgresql://postgres:xxxxx:[YOUR-PASSWORD]@aws-xxxx.pooler.supabase.com:6543/postgres
```

또는:

```
postgresql://postgres:[YOUR-PASSWORD]@db.xxxxx.supabase.co:5432/postgres
```

8.2 비밀번호 치환

연결 문자열 안의 **[YOUR-PASSWORD]**를 프로젝트 생성 때 만든 Database Password로 바꾼다.

기록할 이름:

```
DATABASE_URL=postgresql://...
```

주의:

DATABASE_URL은 절대 공개하면 안 된다.
GitHub에 올리면 안 된다.
브라우저 코드에 넣으면 안 된다.

9. Google OAuth 준비

V1에서는 유저별 작업을 구분해야 하므로 Google 로그인을 붙인다.

얻어야 할 값:

```
GOOGLE_CLIENT_ID=...
```

GOOGLE_CLIENT_SECRET=...

Google OAuth는 Google Cloud Console에서 만든 뒤, Supabase Auth Provider에 입력한다.

10. Supabase에서 Google Provider 화면 열기

먼저 Supabase 쪽에서 Google 로그인 설정 화면을 열고 callback URL을 확인한다.

10.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. 왼쪽 메뉴에서 **Authentication**을 누른다.
3. **Providers** 메뉴를 연다.
4. Provider 목록에서 **Google**을 찾는다.
5. **Google**을 클릭한다.

10.2 callback URL 확인

Google Provider 화면 안에서 **Callback URL** 또는 **Redirect URL**을 확인한다.

보통 이런 모양이다.

`https://xxxxxxxxxxxxxxxxxxxxx.supabase.co/auth/v1/callback`

이 URL은 Google Cloud OAuth Client를 만들 때 **Authorized redirect URIs**에 넣는다.

기록:

`SUPABASE_AUTH_CALLBACK_URL=https://xxxxxxxxxxxxxxxxxxxxx.supabase.co/auth/v1/callback`

이 값은 보통 환경변수로 앱에 넣기보다는 설정 확인용으로만 기록한다.

11. Google Cloud에서 OAuth Client 만들기

11.1 Google Cloud Console 접속

1. 브라우저에서 Google Cloud Console에 접속한다.
2. Google 계정으로 로그인한다.

공식 주소:

`https://console.cloud.google.com/`

11.2 프로젝트 선택 또는 생성

1. 상단 프로젝트 선택 영역을 클릭한다.

2. 기존 프로젝트가 있으면 선택한다.
3. 없으면 **New Project**를 누른다.

추천 프로젝트명:

phoenix-pages-auth

4. 생성 후 해당 프로젝트를 선택한다.

12. Google Auth Platform 설정

Google 로그인은 OAuth Client만 만든다고 끝나지 않는다.

동의 화면, 대상 사용자, 권한 범위도 기본 설정해야 한다.

12.1 Google Auth Platform 이동

1. Google Cloud Console 왼쪽 메뉴 또는 검색창에서 **Google Auth Platform**을 찾는다.
2. **Get started** 또는 설정 화면으로 이동한다.

12.2 Branding 설정

1. **Branding** 메뉴를 연다.
2. App name을 입력한다.

추천:

Phoenix Pages

3. User support email을 선택한다.
4. Developer contact information에 이메일을 입력한다.
5. 저장한다.

주의:

로고나 브랜드 검증은 Google 정책에 따라 시간이 걸릴 수 있다.
초기 개발에서는 최소 정보로 시작하고, 운영 배포 전 브랜드 정보를 정리한다.

12.3 Audience 설정

1. **Audience** 메뉴를 연다.
2. 앱 사용 범위를 선택한다.

일반적으로:

External

3. 테스트 단계라면 Test users에 본인 Google 계정을 추가한다.

주의:

External 앱을 production으로 열기 전에는 Google 검토나 앱 게시 상태를 확인해야 할 수 있다.
입문 테스트 단계에서는 테스트 유저만 추가해서 진행한다.

12.4 Data Access / Scopes 설정

1. **Data Access** 또는 **Scopes** 메뉴를 연다.
2. 기본 프로필과 이메일 권한이 들어가 있는지 확인한다.

Supabase Google 로그인에 필요한 기본 범위:

openid
email
profile

또는 Google 화면에서는 다음처럼 보일 수 있다.

.../auth/userinfo.email
.../auth/userinfo.profile

주의:

불필요한 민감 scope를 추가하지 않는다.
민감 scope가 많아지면 Google 검토가 복잡해질 수 있다.

13. Google OAuth Client ID 만들기

13.1 Clients 메뉴 이동

1. Google Cloud Console에서 **Google Auth Platform**으로 이동한다.
2. **Clients** 메뉴를 연다.
3. **Create client** 또는 **Create OAuth client**를 누른다.

13.2 Application type 선택

Application type:

Web application

Name:

Phoenix Pages Web

13.3 Authorized JavaScript origins 입력

로컬 개발용:

http://localhost:3002

Vercel 배포용:

`https://내-vercel-배포주소.vercel.app`

커스텀 도메인을 쓰면:

`https://phoenix-portal.site`

주의:

origin에는 경로를 넣지 않는다.

예: `https://example.com/auth/callback` 이 아니라 `https://example.com`

13.4 Authorized redirect URIs 입력

Supabase Google Provider 화면에서 확인한 callback URL을 넣는다.

예:

`https://xxxxxxxxxxxxxxxxxxxxx.supabase.co/auth/v1/callback`

로컬 Supabase CLI를 쓸 때는 공식 문서 기준으로 아래 URI를 사용할 수 있다.

`http://127.0.0.1:54321/auth/v1/callback`

하지만 현재 phoenix pages V1은 우선 Supabase hosted project 기준으로 진행한다.

13.5 생성 완료

1. **Create**를 누른다.
2. 생성된 Client ID를 복사한다.
3. Client Secret을 복사한다.

기록:

GOOGLE_CLIENT_ID=복사한_Client_ID
GOOGLE_CLIENT_SECRET=복사한_Client_Secret

주의:

GOOGLE_CLIENT_SECRET은 공개 금지다.
문서, 스크린샷, GitHub에 노출하지 않는다.

14. Supabase에 Google Client ID / Secret 넣기

14.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. **Authentication**으로 이동한다.

3. **Providers**를 연다.
4. **Google**을 클릭한다.

14.2 Google Provider 활성화

1. **Enable Sign in with Google** 또는 **Enable** 토글을 켜다.
2. Google Cloud에서 받은 Client ID를 입력한다.
3. Google Cloud에서 받은 Client Secret을 입력한다.
4. 저장한다.

완료 기준:

Google Provider가 Enabled 상태다.
Client ID와 Client Secret이 저장됐다.
Callback URL이 Google OAuth Client의 Authorized redirect URIs와 일치한다.

15. Supabase Auth URL Configuration 설정

Google 로그인이 끝난 뒤 어느 사이트로 돌아올지 설정한다.

15.1 메뉴 이동

1. Supabase Dashboard에서 프로젝트를 연다.
2. **Authentication**으로 이동한다.
3. **URL Configuration** 또는 **URL Config** 메뉴를 연다.

15.2 Site URL 입력

Vercel 배포 주소를 넣는다.

예:

`https://내-vercel-배포주소.vercel.app`

커스텀 도메인을 쓰면:

`https://phoenix-portal.site`

15.3 Redirect URLs 입력

로컬 개발:

`http://localhost:3002/**`

Vercel 배포:

`https://내-vercel-배포주소.vercel.app/**`

커스텀 도메인:

`https://phoenix-portal.site/**`

주의:

실제 Vercel URL과 Supabase Redirect URL이 맞지 않으면 로그인 후 돌아오지 못한다.

16. 슈퍼 관리자 이메일 정하기

슈퍼 관리자는 전체 유저의 프로젝트와 상태를 볼 수 있는 계정이다.

16.1 기준

슈퍼 관리자 이메일은 Google 로그인에 사용할 이메일과 같아야 한다.

예:

`master@example.com`

16.2 기록할 이름

`SUPER_ADMIN_EMAILS=master@example.com`

여러 명이면:

`SUPER_ADMIN_EMAILS=master@example.com,admin@example.com`

주의:

이 값은 V1에서 관리자 권한 판단에 사용한다.
오타가 있으면 슈퍼 관리자 모드가 열리지 않는다.

17. Vercel에 넣을 환경변수 목록

V1 구현 후 Vercel Project Settings에 넣을 후보 목록이다.

Vercel 메뉴:

Vercel Dashboard
→ Project 선택
→ Settings
→ Environment Variables

권장 입력값:

`NEXT_PUBLIC_SUPABASE_URL=`
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=`
`NEXT_PUBLIC_SUPABASE_ANON_KEY=`
`SUPABASE_SECRET_KEY=`
`SUPABASE_SERVICE_ROLE_KEY=`

```
SUPABASE_STORAGE_BUCKET=phoenix-pages-images
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
SUPER_ADMIN_EMAILS=
DATABASE_URL=
```

주의:

NEXT_PUBLIC_로 시작하는 값은 브라우저에 노출된다.
SUPABASE_SECRET_KEY, SUPABASE_SERVICE_ROLE_KEY, GOOGLE_CLIENT_SECRET, DATABASE_URL에는 NEXT_PUBLIC_을 붙이면 안 된다.

V1에서 실제로 어떤 키 이름을 사용할지는 코딩 시 확정한다.

중복을 줄이기 위해 새 키 체계 또는 레거시 키 체계 중 하나로 정리할 수 있다.

18. 마스터가 코덱스에게 전달할 정보 양식

아래 양식으로 정리하면 V1 작업을 바로 시작할 수 있다.

실제 비밀 키를 채팅에 붙여넣기 부담스러우면, 마스터가 직접 Vercel 환경변수에 입력하고 코덱스에게는 **입력 완료** 라고 알려줘도 된다.

```
[Supabase]
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SECRET_KEY=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_STORAGE_BUCKET=phoenix-pages-images
```

```
[Google OAuth]
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
```

```
[Admin]
SUPER_ADMIN_EMAILS=
```

```
[Optional]
DATABASE_URL=
```

```
[Deploy]
Vercel Project URL=
Production Domain=
Local URL=http://localhost:3002
```

19. 마스터가 직접 해야 하는 일과 코덱스가 할 일

19.1 마스터가 직접 해야 하는 일

1. Supabase 프로젝트 생성
2. Supabase Project URL 확인
3. Supabase 공개용 키 확인
4. Supabase 서버용 비밀 키 확인

5. Storage bucket 생성
6. Google OAuth Client ID / Secret 생성
7. Supabase Google Provider에 Client ID / Secret 입력
8. Supabase URL Configuration 설정
9. 슈퍼 관리자 이메일 결정
10. Vercel 환경변수 입력 또는 코덱스에게 전달

19.2 코덱스가 할 일

1. Supabase client 설치 및 설정
2. Google 로그인 UI 추가
3. 로그인 세션 처리
4. Supabase Database 테이블 설계
5. RLS 정책 작성
6. Storage upload API 작성
7. 생성 이미지를 Storage에 저장
8. DB에 프로젝트와 섹션 정보 저장
9. 유저별 작업 목록 구현
10. 슈퍼 관리자 모드 구현
11. 기존 V0 브라우저 저장 fallback 유지
12. 배포 빌드 검증

20. 준비 완료 체크리스트

아래 항목이 모두 준비되면 V1 작업을 시작할 수 있다.

- [] Supabase 프로젝트가 있다.
- [] **NEXT_PUBLIC_SUPABASE_URL**을 확보했다.
- [] 공개용 Supabase key를 확보했다.
- [] 서버용 Supabase secret 또는 service role key를 확보했다.
- [] Storage bucket **phoenix-pages-images**를 만들었다.
- [] Google Cloud 프로젝트를 만들었다.
- [] Google OAuth Client ID를 확보했다.
- [] Google OAuth Client Secret을 확보했다.
- [] Google OAuth Redirect URI에 Supabase callback URL을 넣었다.
- [] Supabase Google Provider를 활성화했다.
- [] Supabase Site URL과 Redirect URLs를 설정했다.
- [] 슈퍼 관리자 이메일을 정했다.

- [] Vercel 환경변수 입력 준비가 됐다.

21. 공식 참고 문서

Supabase API Keys:

<https://supabase.com/docs/guides/getting-started/api-keys>

Supabase Google Login:

<https://supabase.com/docs/guides/auth/social-login/auth-google>

Supabase Storage Buckets:

<https://supabase.com/docs/guides/storage/buckets/creating-buckets>

Google OAuth Clients:

<https://support.google.com/cloud/answer/15549257>

22. 핵심 주의사항

1. **SUPABASE_SECRET_KEY**, **SUPABASE_SERVICE_ROLE_KEY**, **GOOGLE_CLIENT_SECRET**, **DATABASE_URL**은 절대 공개하지 않는다.
2. **NEXT_PUBLIC_** 접두사는 공개되어도 되는 값에만 붙인다.
3. Storage bucket은 운영형 V1에서는 Private을 우선 권장한다.
4. 슈퍼 관리자 이메일은 Google 로그인 이메일과 정확히 일치해야 한다.
5. V1 작업 전에 V0 브라우저 저장 기능은 fallback으로 남겨두는 것이 좋다.
6. 기존 유저가 만든 브라우저 저장 작업은 V1 전환 시 자동으로 클라우드로 올라가지 않는다. 별도 마이그레이션 기능이 필요하다.
7. Google OAuth Redirect URL이 하나라도 틀리면 로그인 후 앱으로 돌아오지 못한다.
8. RLS 정책 없이 Supabase DB를 열면 유저 데이터가 섞일 수 있으므로 V1 구현 때 반드시 유저별 정책을 넣는다.

23. MCP 보조 방식으로 준비하는 방법

V1 준비는 전부 수동으로 해도 되지만, Supabase MCP를 사용하면 코덱스가 Supabase 프로젝트 상태를 직접 확인하고 SQL, RLS, Storage 관련 작업을 더 안정적으로 도울 수 있다.

단, MCP는 모든 것을 대신하는 만능 자동화가 아니다.

권장 구조:

Supabase DB / RLS / Storage 확인 = MCP 보조 가능
 Google OAuth Client 생성 = Google Cloud Console에서 마스터 확인 권장
 Supabase Google Provider Client Secret 입력 = 마스터 직접 입력 권장

공식 Supabase MCP 문서:

<https://supabase.com/docs/guides/ai-tools/mcp>

공식 Supabase MCP GitHub:

<https://github.com/supabase/mcp>

24. Supabase MCP로 할 수 있는 일

Supabase MCP는 V1 작업에서 아래 영역에 유용하다.

1. Supabase 프로젝트 목록 확인
2. 특정 프로젝트 정보 확인
3. Project URL 확인
4. 공개용 key 확인
5. Database table 목록 확인
6. SQL 실행
7. migration 적용
8. RLS 정책 확인
9. Storage bucket 목록 확인
10. 로그 확인
11. TypeScript 타입 생성
12. Edge Function 관련 작업

V1에서는 특히 아래 작업에 적합하다.

profiles/projects/project_sections/admin_users 테이블 생성
RLS 정책 적용
Storage bucket 존재 여부 확인
이미지 저장 경로 정책 확인
프로젝트 저장 API 오류 로그 확인

25. Supabase MCP 연결 시 권장 설정

Supabase MCP remote endpoint:

<https://mcp.supabase.com/mcp>

특정 프로젝트로 범위를 제한하려면 **project_ref**를 붙인다.

https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF

처음에는 읽기 전용으로 연결하는 것을 권장한다.

https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF&read_only=true

Storage 기능까지 확인하려면 feature group을 추가한다.

`https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF&features=database,docs,storage`

초기 확인용 추천:

`https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF&read_only=true&features=database,docs,storage`

실제 SQL과 migration을 적용해야 할 때만 read-only를 해제한다.

주의:

처음부터 전체 권한으로 연결하지 않는다.

수강생은 반드시 자기 Supabase 프로젝트에만 연결한다.

마스터의 Supabase project_ref나 token을 수강생에게 공유하지 않는다.

26. MCP 방식에서도 마스터가 직접 해야 하는 일

Supabase MCP를 사용해도 아래 작업은 마스터가 직접 확인하는 것이 안전하다.

1. Supabase 계정 로그인
2. MCP 연결 승인
3. project_ref 확인
4. Google Cloud OAuth Client 생성
5. Google OAuth Client Secret 확인
6. Supabase Google Provider에 Client ID / Secret 입력
7. Supabase Auth Site URL / Redirect URLs 입력
8. Vercel 환경변수에 secret 입력
9. 실제 key/token이 문서와 스크린샷에 노출되지 않았는지 확인

특히 아래 값은 MCP나 코텍스 채팅창에 직접 붙여넣지 않는 것이 좋다.

```
SUPABASE_SECRET_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
GOOGLE_CLIENT_SECRET=  
DATABASE_URL=
```

27. Google Provider 설정과 MCP의 한계

V1의 Google 로그인에는 **Supabase Auth의 Google Provider**를 사용한다.

이 작업은 두 시스템을 동시에 다룬다.

Google Cloud Console
→ OAuth Client ID / Secret 생성

Supabase Dashboard

- Authentication
- Providers
- Google
- Client ID / Secret 입력

Google Cloud에도 MCP/gcloud 기반 자동화 도구가 있을 수 있지만, OAuth Client Secret은 보안 민감값이다.

따라서 수강생용 권장 방식은 아래와 같다.

Google OAuth Client 생성 = 마스터가 Google Cloud Console에서 직접
Supabase Google Provider 활성화 = 마스터가 Supabase Dashboard에서 직접
코덱스 역할 = 설정값 이름, redirect URL, 코드 연동, 오류 확인 지원

Google Cloud MCP 참고:

<https://docs.cloud.google.com/mcp/overview>

Google OAuth 설정은 최종 집필 또는 강의 전 공식 화면을 다시 확인한다.